

FleetIntel: AI-Integrated Telemetry Pipeline

FleetIntel is an end-to-end IoT ecosystem that transforms raw vehicle data into natural language mechanical diagnostics. Instead of just showing a dashboard of gauges, this system uses a "Detective" engine to analyze engine health in real-time, considering both internal sensor data and external environmental factors.

The Core Logic

Most fleet trackers use static "red-line" thresholds (e.g., alert if Temp > 100°C). FleetIntel takes a different approach:

- **Adaptive Baselines:** The system calculates Z-scores based on a vehicle's historical performance. It learns what "normal" looks like for a specific engine before it starts flagging anomalies.
- **Data Fusion:** Using the **Open-Meteo API**, the system pulls live weather and GPS elevation. This allows the AI to distinguish between a genuine cooling issue and a car simply working hard to climb a steep hill in 30°C heat.
- **Natural Language Reporting:** Raw PID codes are fed into **GPT-4o** to generate a human-readable diagnosis sent directly to the fleet manager via Telegram.

Tech Stack

- **Hardware:** Teltonika FMC003 (LTE Cat M1/NB-IoT)
- **Ingestion:** Flespi (MQTT Broker / Protocol Gateway)
- **Database:** PostgreSQL + TimescaleDB (Optimized for time-series telemetry)
- **Intelligence:** OpenAI API (GPT-4o), Open-Meteo API (Weather/Elevation)
- **Backend:** Python 3.10+ (Pandas, Paho-MQTT, Psycopg2)
- **Infrastructure:** Ubuntu Server (Systemd for service resilience)

Project Structure

- `src/logger.py`: Handles the MQTT stream from Flespi and maps raw JSON hex to our PostgreSQL hypertable.
 - `src/guardian.py`: The "Brain." It runs the anomaly detection loop, fetches environmental context, and manages the AI diagnostic flow.
 - `db/schema.sql`: The relational blueprints. Uses an EAV (Entity-Attribute-Value) model to handle any number of OBDII PIDs without needing table migrations.
 - `.env.example`: A template for the required API keys and database credentials.
-

Self-Healing & Integrity

To ensure the system stays production-ready, I built a custom `/status` command into the Admin Telegram bot. It performs a real-time audit of:

1. **Schema Integrity:** Checks if all required tables and columns exist.
2. **Service Health:** Verifies the status of the background systemd services.
3. **Data Velocity:** Reports the number of pings received in the last 5 minutes to ensure the hardware is still talking to the cloud.